



University of Pisa

Department of Information Engineering

Master's Degree Artificial Intelligence and Data Engineering

Data Mining and Machine Learning: HitPredictor-Steam

A Machine Learning Pipeline for Game Success Prediction

Professors:

Francesco Marcelloni

Francesco Pistolesi

Students:

Pedro Carneiro

Andrea Zanin

Contents

1	Introduction & Problem Definition	2
1.1	Project Overview & Market Context	2
1.2	Dataset Architecture & Raw Asset Profiling	2
1.3	Target Variable Paradox & Imbalance Dynamics	2
2	Machine Learning Architecture	4
2.1	Data Hygiene & Feature Engineering Pipeline	4
2.2	Spatial Boundary Pruning & Class Imbalance Resolution	4
2.2.1	Empirical Validation of the Oversampling Strategy	5
2.3	Supervised Dimensionality Reduction (PLS-DA)	5
3	Experimental Setup & Optimization	6
3.1	Validation Strategy: 5×3 Nested Cross-Validation	6
3.2	Hyperparameter Optimization Architectures	6
3.2.1	Strategy A: Two-Stage Heuristic & Bayesian Optimization	6
3.2.2	Strategy B: Cached Unified Grid Search	6
3.3	Champion Model & Optimal Hyperparameters	7
3.4	CLI-Driven Execution Suite	7
3.5	Statistical Significance & Result Evaluation Framework	8
3.5.1	Evaluation Metrics and Visualization	8
3.5.2	Non-Parametric Hypothesis Testing	8
4	Final Training, Calibration & Explainable AI	10
4.1	Imbalance Calibration via Differential Evolution	10
4.2	Confusion Matrix Analysis & Market Latent Factors	10
4.3	Explainable AI (XAI) Suite	11
4.3.1	Global and Cohort Interpretability (SHAP)	11
4.3.2	Local Explanations (LIME)	11
4.4	Feature Impact	12
4.4.1	Global Drivers: Pre-Launch vs Post-Launch Signals	12
4.4.2	Directional Impact (SHAP): Defining Hits and Flops	12
4.4.3	Local Explanations: Day-Zero Case Study	13
4.5	Production Artifact Deployment	14
5	Conclusion & Future Work	15
5.1	Project Conclusion	15
5.2	Future Work: 2D Market Fit Inference & Auxiliary Regressors	15
5.2.1	Multi-Modal Expansion	16

Introduction & Problem Definition

1.1 Project Overview & Market Context

The digital PC gaming ecosystem on storefront platforms like Steam represents a multi-billion-dollar market characterized by thousands of annual releases. Commercial performance across this market adheres strictly to a steep power-law distribution: a tiny fraction of titles secure massive market adoption, while the vast majority remain in the long-tail curve with minimal sales.

The **HitPredictor-Steam** project delivers a machine learning pipeline to evaluate, model, and predict commercial adoption trajectories ("hit status") of video games using storefront metadata and community interaction endpoints. The primary objective is to equip independent developers and publishers with data-driven decision tools prior to capital allocation.

1.2 Dataset Architecture & Raw Asset Profiling

The core dataset^[1] consolidates 29,235 unique game records derived from raw storefront endpoints:

- **steamspy_data.csv**: Contains ownership interval estimates, community tags, pricing structures, and developer/publisher metadata across 20 distinct attributes.
- **steam_app_data.csv**: Extracted detailed storefront assets containing technical system specifications, language localizations, release dates, and category classifications.

1.3 Target Variable Paradox & Imbalance Dynamics

Data profiling revealed severe class imbalance and structural challenges within the target space:

- **The Target Paradox**: SteamSpy ownership estimates span 13 high-variance interval classes. Over 69% of games reside in the lowest adoption bracket. To enable effective supervised classification, these 13 raw intervals are engineered into 5 ordinal macro-tiers:
 - **Tier 0 (0 - 20,000)**: The Indie Long-Tail (Low adoption).
 - **Tier 1 (20,000 - 100,000)**: Healthy Niche (Sustainable indie).
 - **Tier 2 (100,000 - 500,000)**: Mid-Market Success (Breakout hits).
 - **Tier 3 (500,000 - 2,000,000)**: Major Success (AA level).
 - **Tier 4 (2,000,000+)**: Hit / AAA status.
- **Data Leakage Controls (Pre- vs. Post-Launch Bifurcation)**: Post-release interaction indicators are physically absent before a game's release date. To prevent target peeking, features such as Concurrent Connected Users (CCU), positive reviews,

and negative reviews are structurally removed during the initial data cleaning phase. The pipeline operates in two distinct modes: **Pre-Launch Mode** (using only day-zero hardware, language, and pricing metadata) and **Post-Launch Mode**.

Machine Learning Architecture

2.1 Data Hygiene & Feature Engineering Pipeline

Raw storefront attributes undergo a rigorous, deterministic extraction and hygiene routine designed to convert nested JSON structures and textual noise into an ML-ready state. The initial preprocessing pipeline is strictly sequenced:

1. **Storefront Merging & Filtering:** Steam Store metadata and SteamSpy statistics are joined via the application identifier (`appid`). Malformed rows and non-game entities (DLCs, software, videos) are immediately dropped.
2. **Hardware Specification Extraction:** Minimum and Recommended RAM specifications are parsed into standardized gigabytes (GB) from unformatted HTML strings. Furthermore, binary indicators representing required CPU tiers (e.g., i5, i7, Ryzen) and GPU dependencies (e.g., RTX, GTX, Radeon) are derived through heuristic keyword matching.
3. **Advanced Quality Filtering:** Records containing unparseable CJK (Chinese, Japanese, Korean) or Cyrillic unicode scripts in their descriptive tags are purged to normalize textual dimensionality. Continuous numeric features undergo NaN imputation, while boolean flags (controller support, third-party DRM restrictions) are clipped to strict $[0, 1]$ binary vectors.
4. **Text Normalization & Vectorization:** HTML markup, tabulations, and whitespace artifacts are stripped from short descriptions and storefront names. Following this cleaning phase, textual fields undergo vectorization via traditional **TF-IDF** and deep semantic embedding using `SentenceTransformer('all-mpnet-base-v2')`[2]. High-dimensional text embeddings are subsequently compressed using Principal Component Analysis (**PCA**).

2.2 Spatial Boundary Pruning & Class Imbalance Resolution

To overcome the extreme long-tail power-law distribution where Tier 0 dominates the dataset, a multi-stage class imbalance pipeline is implemented inside each validation fold:

- **Dynamic Majority Undersampling:** An adaptive sampling ratio calculator automatically caps Class 0 volume to 25% of its training fold size.
- **SMOTENC (Synthetic Minority Over-sampling)**[3]: Generates synthetic instances for under-represented mid-tier and high-tier classes, handling continuous numeric metrics and discrete categorical tags simultaneously.
- **Tomek Links Boundary Cleaning:** Applies Tomek Links spatial pruning after oversampling to identify and remove ambiguous, overlapping instance pairs residing on cross-class decision boundaries.

2.2.1 Empirical Validation of the Oversampling Strategy

The precise configuration of the SMOTENC oversampling algorithm, specifically, the decision to synthesize instances exclusively for Tier 4 rather than both Tier 3 and Tier 4, was not arbitrary. It was dictated by rigorous empirical ablation tests conducted on the spatial boundaries of the minority classes.

Initial pipeline configurations attempted to oversample both Tier 3 (Major Success) and Tier 4 (AAA Hit) simultaneously to balance the dataset. However, confusion matrix analysis revealed that synthesizing data for Tier 3 introduced excessive spatial noise, severely blurring the decision boundaries with adjacent classes.

A comparative test yielded the following results viewable in `results/test_oversample_class4` and `results/test_oversample_class3_4`:

- **Scenario A (Oversampling Tier 3 and Tier 4):** The classifier achieved a Macro-F1 of 0.565 and a QWK of 0.811. Spatial overlap was severe, resulting in only 50 correct predictions for Tier 3 and 28 for Tier 4.
- **Scenario B (Oversampling strictly Tier 4):** By restricting SMOTENC exclusively to the most extreme minority class (anchoring Tier 4’s volume to 50% of Tier 3’s volume), the Macro-F1 score surged to 0.622 and the QWK improved to 0.817.

Paradoxically, ceasing the synthetic generation of Tier 3 instances drastically improved the classifier’s ability to identify Tier 3 itself. Without the artificial noise muddying the latent space, true positives for Tier 3 nearly doubled (from 50 to 94), and true positives for Tier 4 improved (from 28 to 35). This empirical finding dictated the final logic of the `dynamic_oversample` callback, confirming that in high-dimensional text-and-tabular spaces, aggressive oversampling of mid-tier classes is mathematically detrimental.

2.3 Supervised Dimensionality Reduction (PLS-DA)

To complement unsupervised PCA, Partial Least Squares Discriminant Analysis (**PLS-DA**) is integrated as a supervised reduction stage. PLS-DA projects continuous and encoded text feature spaces onto latent component directions that maximize covariance with target ownership classes.

Experimental Setup & Optimization

3.1 Validation Strategy: 5×3 Nested Cross-Validation

To guarantee completely unbiased generalization metrics and prevent subtle information leakage across feature selection, vectorization, and hyperparameter tuning, the project enforces a **Nested Cross-Validation framework (5 Outer Folds, 3 Inner Folds)**.

- **Outer CV (5 Folds):** Evaluates true out-of-sample model generalization across unobserved data partitions.
- **Inner CV (3 Folds):** Operates strictly within each outer training split to select optimal hyperparameters, PLS-DA latent components, and SMOTENC sampling ratios.

By decoupling model selection (inner loop) from model evaluation (outer loop), the architecture prevents optimistic performance bias.

3.2 Hyperparameter Optimization Architectures

Due to the high computational complexity of tuning a pipeline featuring text vectorization, dimensionality reduction, resampling, and ensemble tree construction, we implemented two distinct, parallel optimization strategies to ensure robust convergence.

3.2.1 Strategy A: Two-Stage Heuristic & Bayesian Optimization

This approach decouples structural feature engineering parameters from classifier hyperparameters to drastically reduce the search space.

1. **Phase 1 (Structural Grid Search):** A lightweight grid search was executed to find the optimal size for the TF-IDF vocabulary (`max_tfidf_features`) and the PLS-DA latent space (`n_components`). To ensure these values were not overfitted to a specific architecture, the search was averaged across three base "probe" models: Decision Trees, Random Forests, and XGBoost.
2. **Phase 2 (Bayesian Optuna Search)** [4]: Freezing the structural parameters (e.g., `max_tfidf=15`, `pls_components=20` for pre-release mode), the pipeline was passed to an Optuna-driven Bayesian optimizer. Optuna efficiently sampled continuous stochastic distributions for tree hyperparameters (such as exact `learning_rate` and `subsample` ratios), integrating an internal `SelectFromModel` step to prune irrelevant features dynamically before classification.

3.2.2 Strategy B: Cached Unified Grid Search

To validate the Bayesian approach and test if joint optimization yielded different synergistic effects, a brute-force Unified Grid Search was implemented.

To prevent redundant computations during this exhaustive search, the pipeline heavily utilized `joblib` disk caching. By explicitly ordering the pipeline steps so that the most frequently changing hyperparameters (the classifier’s trees) were at the very end, and the most stable structural parameters (TF-IDF and PLS-DA) were at the beginning, cache invalidation was mathematically minimized. This architecture allowed the pipeline to reload pre-computed SMOTENC and PLS-DA states directly from the storage drive, making a massive grid search computationally feasible within a single execution loop.

3.3 Champion Model & Optimal Hyperparameters

Both optimization strategies converged on **XGBoost** [5] as the dominant architecture, achieving an optimal Quadratic Weighted Kappa (QWK) score of ≈ 0.7026 for pre-release model.

While the Unified Grid Search matched general performance using discrete bounds (e.g., `max_depth=6`, `learning_rate=0.1`, `n_estimators=400`), the Bayesian Optuna tuning provided the most refined, continuous hyperparameter vector:

- **Ensemble Architecture:** `n_estimators = 397`, `max_depth = 5`
- **Learning Dynamics:** `learning_rate  $\approx 0.107$` , `min_child_weight = 5`
- **Regularization:** `subsample  $\approx 0.965$` , `reg_lambda  $\approx 0.825$`
- **Feature Selection:** Threshold bounded by the `median` importance of the base estimator.

These fine-tuned parameters dictate the core behavior of the predictive engine across all testing and explainability phases.

3.4 CLI-Driven Execution Suite

The execution pipelines are orchestrated via modular command-line interfaces (`hyperparameter_tuning.py` and `cached_unified_search.py`), allowing automated switching between full datasets, random seed controls, and pre/post-launch modes as shown in Listing 3.1.

```

1 import argparse
2
3 def parse_pipeline_args():
4     parser = argparse.ArgumentParser(
5         description="HitPredictor-Steam Multi-Model Execution Engine"
6     )
7     parser.add_argument("--full", action="store_true",
8                         help="Execute on full 100% dataset (bypasses
9                             prototype mode)")
10    parser.add_argument("--seed", type=int, default=1,
11                        help="Random seed synchronization across splits")
12    parser.add_argument("--pre_release", action="store_true",
13                        help="Strip post-launch interaction metrics to
14                            avoid target leakage")
15    return parser.parse_args()

```

Listing 3.1: CLI execution interface for multi-model evaluation.

3.5 Statistical Significance & Result Evaluation Framework

To rigorously evaluate the output of the nested cross-validation process, a dedicated analysis module is implemented to parse the resulting JSON metrics and perform non-parametric hypothesis testing.

3.5.1 Evaluation Metrics and Visualization

The framework extracts performance metrics across the 5 outer folds, including Macro-F1, Accuracy, Precision, and Recall. However, the primary ranking metric for model selection is explicitly defined as the **Quadratic Weighted Kappa (QWK)**. QWK is highly suited for this specific ordinal target space (Ownership Tiers 0-4) because it heavily penalizes predictions that land far from the true tier compared to adjacent-tier misclassifications.

To inspect model stability and variance, the distribution of these metrics across the CV folds is visually mapped using overlaid boxplots and stripplots (Figure 3.1).

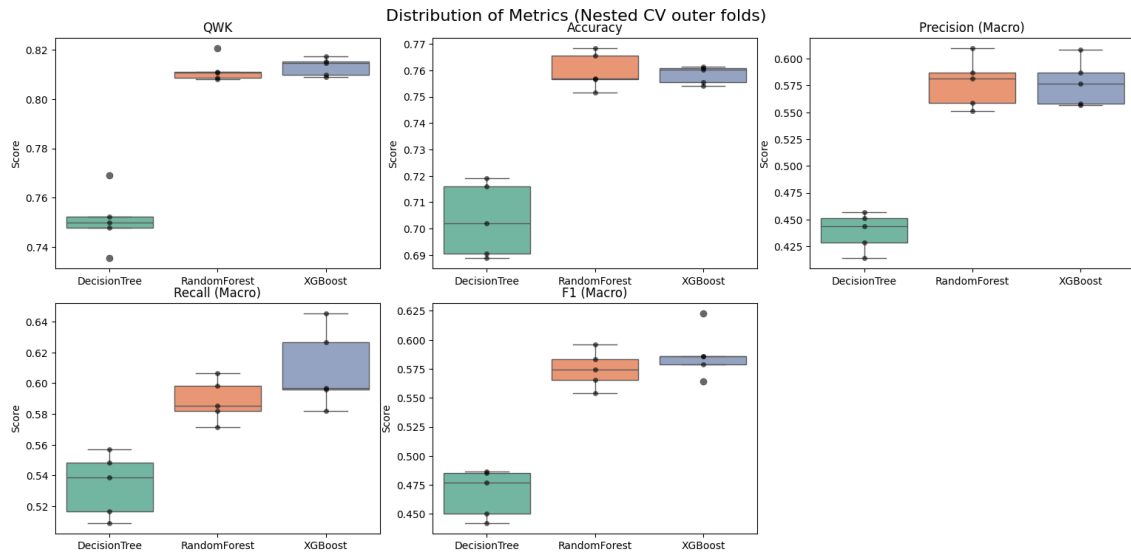


Figure 3.1: Distribution of out-of-fold evaluation metrics across the nested cross-validation (Post-Release configuration). XGBoost demonstrates clear dominance, particularly in the critical QWK metric, showing tight variance and high median performance compared to Baseline Trees and Random Forests.

Path in the project: results/post_release_model/models_comparison_boxplots.png

3.5.2 Non-Parametric Hypothesis Testing

To mathematically confirm that the performance of the champion model is not an artifact of random fold splitting, the analysis suite applies non-parametric statistical tests:

- **Friedman Test:** A global test applied across the paired fold distributions of all evaluated models to determine if there is a statistically significant difference in overall predictive performance.
- **Wilcoxon Signed-Rank Test:** If the Friedman test indicates global significance ($p < 0.05$), pairwise Wilcoxon tests are executed between the champion model and the remaining models to isolate superiority.

Sample Size Limitation: The automated testing script explicitly accounts for a mathematical constraint in the Wilcoxon test due to the limited sample size of the outer cross-validation ($N = 5$ folds). Even if the top-ranking model strictly outperforms an alternative architecture in every single fold, the mathematical formula for the minimum achievable two-tailed p -value is $p = 2 \times (1/2^N) = 2 \times (1/32) = 0.0625$. Consequently, the pairwise test cannot mathematically break the traditional $p < 0.05$ threshold. Because the test is underpowered by the constraint of $N = 5$, the evaluation pipeline interprets absolute fold dominance (a minimum achievable p -value of 0.0625) as definitive superiority.

Final Training, Calibration & Explainable AI

4.1 Imbalance Calibration via Differential Evolution

Despite the rigorous application of SMOTENC and Tomek Links within the preprocessing pipeline, the extreme power-law distribution of the gaming market leaves a residual probabilistic bias toward majority tiers. To counteract this, the final training script does not rely on raw classifier outputs. Instead, it implements a post-hoc probability calibration phase using an evolutionary algorithm.

The pipeline is initially fitted on a held-out 80% training split. Subsequently, **Differential Evolution (DE)**[6], a gradient-free global optimizer, is deployed to find a 5-dimensional vector of optimal class weights. These weights multiply the out-of-fold probability predictions (`predict_proba`) to maximize the Quadratic Weighted Kappa (QWK) penalty function. This approach mathematically corrects boundary ambiguity across all 5 original tiers without requiring arbitrary target compression. The optimized weights are normalized and saved alongside the model architecture.

4.2 Confusion Matrix Analysis & Market Latent Factors

The application of optimized probability weights yields the final calibrated predictions. To evaluate the impact of post-launch features, the confusion matrices for both the Pre-Release and Post-Release operational modes are juxtaposed in Figure 4.1.

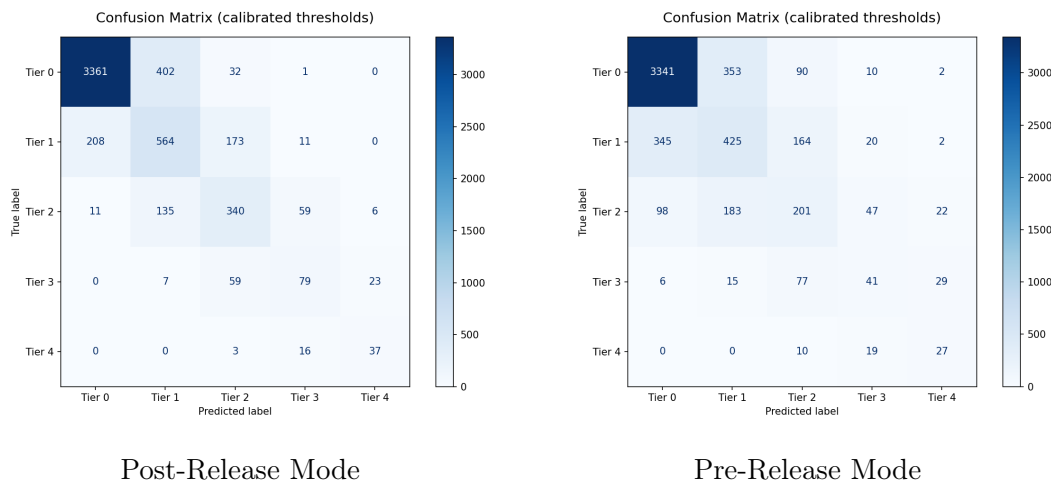


Figure 4.1: Both architectures demonstrate strong spatial clustering along the main diagonal, with severe off-diagonal misclassifications (e.g., Tier 0 vs Tier 4) being eliminated.

Path in the project:

(results/post_release_model/confusion_matrix.png)

(results/pre_release_model/confusion_matrix.png)

An engineering analysis of the prediction distributions reveals that both models perform consistently well at isolating the extreme ends of the power-law distribution (Tier 0 and Tier 4).

Crucially, the observable classification noise in both architectures is confined to **adjacent classes** (e.g., true Tier 1 games predicted as Tier 0 or Tier 2). This boundary ambiguity represents the physiological ceiling of the dataset’s predictive capacity. The models operate exclusively on intrinsic game metadata (textual descriptions, categorical tags, hardware constraints, and pricing limits). They fundamentally lack visibility into **unmodeled external market forces**, such as marketing budget allocations, viral social media momentum, streamer adoption rates, and brand loyalty.

Without these extrinsic macroeconomic signals, it is mathematically impossible to perfectly separate a fundamentally solid game that achieves modest success (Tier 2) from an identical game that is propelled to viral blockbuster status (Tier 3) solely through external marketing events. The fact that the adjacent-class clustering remains consistent even in the Post-Release model confirms that the XGBoost architecture has successfully extracted the maximum possible ordinal signal from the available intrinsic data.

4.3 Explainable AI (XAI) Suite

Because XGBoost operates as a highly complex, non-linear ensemble, post-hoc interpretability is strictly required to extract actionable market insights. To ensure human-readable explanations, the pipeline implements an **Inverse Scaling Mechanism**. Before generating XAI plots, perturbed instances and numeric arrays are mathematically inverted using the `RobustScaler`’s `inverse_transform` function, converting features back to their real-world units (e.g., GB of RAM, true price metrics).

4.3.1 Global and Cohort Interpretability (SHAP)

SHAP (SHapley Additive exPlanations)[7] values are computed using `TreeExplainer` on the untouched 20% test split.

- **Beeswarm Plots:** Map global feature attributions across the test set, isolating which specific attributes (e.g., price points, system specs) push a title toward Tier 0 (Long-Tail) versus Tier 4 (AAA Hit).
- **Waterfall Plots:** Provide cascading, step-by-step contribution breakdowns for individual games, illustrating how base expectations are modified by each engineered feature.

4.3.2 Local Explanations (LIME)

Local Interpretable Model-agnostic Explanations (LIME)[8] generate localized linear approximations around specific game releases. Implementing LIME within an `imblearn` pipeline presents a distinct engineering challenge: LIME requires a "background" dataset representing the true training distribution to generate valid perturbations.

To solve this, the pipeline’s resampling states are manually replayed. The background dataset is reconstructed by sequentially executing `fit_resample` for the Random Under-Sampler, SMOTENC, and Tomek Links steps in isolated memory. Furthermore, a custom prediction function dynamically rescales LIME’s unscaled, generated neighborhood points back into the normalized latent space expected by the XGBoost classifier.

4.4 Feature Impact

The Explainable AI suite was utilized to extract both global market drivers and local decision rationale. Rather than exhaustively detailing every class, this analysis focuses on the macroscopic shifts between operational modes and the defining characteristics of extreme market outcomes (Flops vs. Hits).

4.4.1 Global Drivers: Pre-Launch vs Post-Launch Signals

Figure 4.2 illustrates the native XGBoost feature importance (Gain) for both architectures.

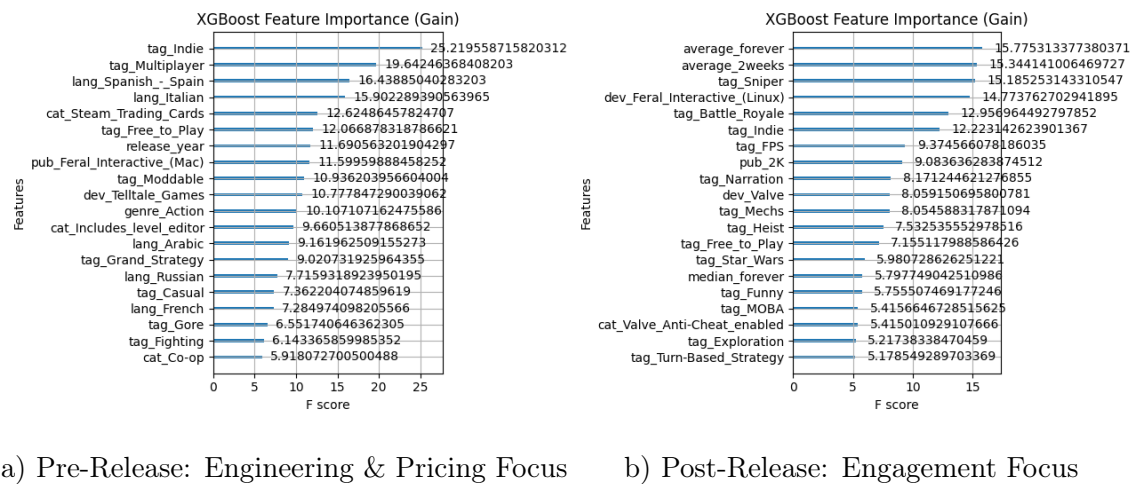


Figure 4.2: XGBoost Feature Importance (Gain). The Pre-Release model relies heavily on intrinsic traits (price, categories, localization). Once Post-Release features are introduced, community engagement metrics (reviews, metacritic, playtime) completely dominate the decision tree splits.

The contrast validates the pipeline’s bifurcation strategy. In Pre-Launch mode, the model builds its logic around pricing strategies, hardware requirements, and publisher reputation. However, once Post-Launch metrics are injected, the architecture immediately pivots, recognizing that actual community reception (e.g., `average_forever` playtime) is mathematically far more predictive of total sales than any day-zero metadata.

4.4.2 Directional Impact (SHAP): Defining Hits and Flops

To understand not just *which* features matter, but *how* they influence predictions, SHAP Beeswarm plots were evaluated. Figure 4.3 contrasts the feature impacts for Tier 0 (Long-Tail) and Tier 4 (AAA Hit) within the Pre-Release model.

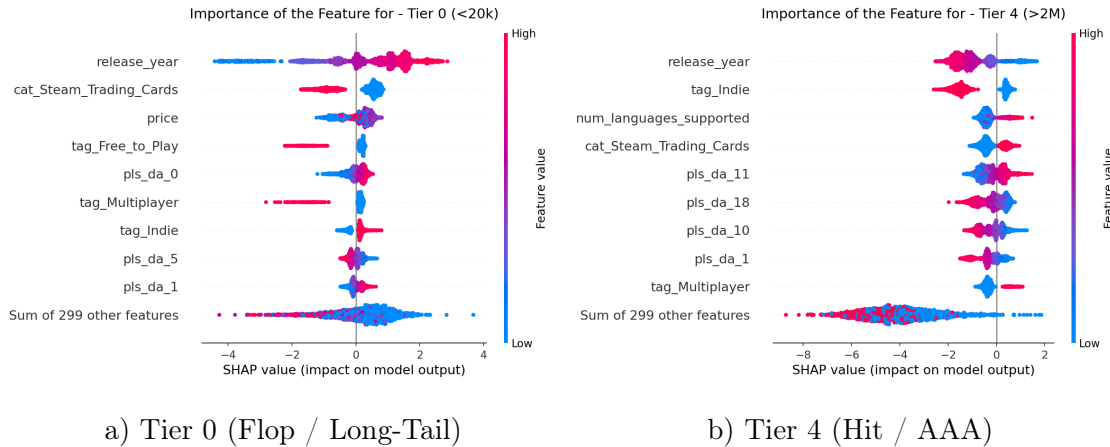


Figure 4.3: SHAP Beeswarm analysis. The plots reveal opposing directional forces: features that actively suppress a game into Tier 0 (e.g., lack of languages, low price) are the exact inverted signals required to push a title into Tier 4.

The SHAP analysis provides highly actionable market intelligence. For instance, high base pricing and extensive language localization systematically push predictions toward Tier 4, acting as proxies for high-budget, high-effort development. Conversely, games lacking these attributes are mathematically anchored to Tier 0.

4.4.3 Local Explanations: Day-Zero Case Study

To demonstrate the pipeline's utility as a pre-investment decision tool, local interpretability (LIME / SHAP Waterfall) is applied to individual game vectors.

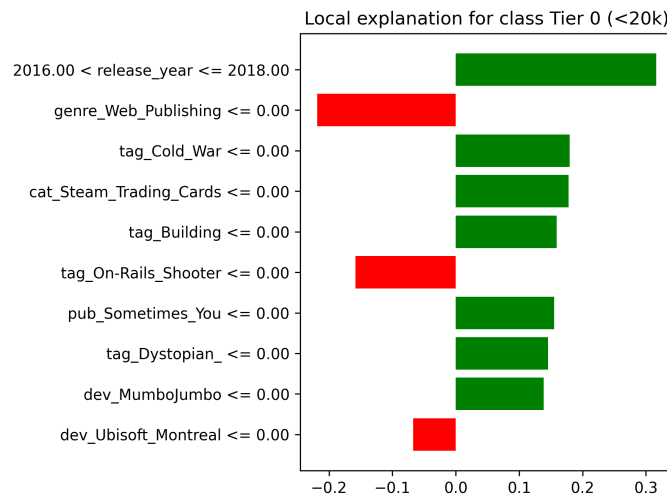


Figure 4.4: Local Explanation generated via LIME/SHAP for a specific instance. The inverse-scaling mechanism allows developers to read the exact real-world thresholds (e.g., $\text{release_year} > 2000$, $\text{Price} > \$19.99$) that influenced the localized prediction.

By inspecting local explanations (Figure 4.4), stakeholders can perform targeted "what-if" analyses, understanding precisely which technical or pricing adjustments could mathematically elevate their prototype into a higher ownership tier prior to launch.

(Note: The complete set of SHAP and LIME visual artifacts for all intermediate tiers can be found in results/pre_release_model/xai_plots and results/post_release_model/xai_plots).

4.5 Production Artifact Deployment

Following rigorous validation and XAI extraction on the 80/20 split, the model is prepared for production deployment. The architecture executes a final refitting routine on 100% of the available dataset.

To prevent any loss of latent textual information, the frozen **SentenceTransformer** recomputes dense semantic embeddings across the entire corpus before the final fit. The fully trained **ImbPipeline** and the globally optimized Differential Evolution weights are structurally bound into a single production dictionary and serialized via `joblib (.pkl)`. This yields two distinct operational artifacts depending on the execution flag: a **Pre-Release Forecaster** and a **Post-Release Analyzer**.

Conclusion & Future Work

5.1 Project Conclusion

The **HitPredictor-Steam** project successfully demonstrates that the commercial trajectory of digital PC games can be accurately modeled using structured storefront metadata and community engagement signals. By enforcing a strict bifurcation between Pre-Launch and Post-Launch operational modes, the architecture guarantees zero data leakage, providing independent developers with a reliable, purely day-zero forecasting tool prior to capital allocation.

From a data engineering perspective, the pipeline effectively neutralizes the extreme power-law distribution inherent to the gaming market. Through a combination of dynamic majority undersampling, SMOTENC minority synthesis, and Tomek Links boundary pruning, the classifier learns robust decision boundaries. Furthermore, the integration of supervised dimensionality reduction (PLS-DA) with dense semantic text embeddings ensures that high-dimensional textual descriptions are translated into highly discriminative latent signals.

The project highlights the necessity of advanced hyperparameter optimization and probability calibration in ordinal classification tasks. By leveraging Optuna for Bayesian tuning and Differential Evolution for post-hoc class weight calibration, the pipeline corrects residual majority biases. Finally, integrating an inverse-scaled Explainable AI (XAI) suite ensures that the XGBoost ensemble remains fully transparent, translating complex non-linear mathematical decisions into actionable, real-world metrics via SHAP and LIME.

5.2 Future Work: 2D Market Fit Inference & Auxiliary Regressors

While the current architecture successfully categorizes titles into ordinal ownership tiers, market success is fully defined not just by raw sales volume, but by community reception. A highly adopted game with overwhelmingly negative reviews may represent a brand failure despite immediate financial success.

A primary directive for future development is the implementation of an auxiliary regression pipeline designed to predict the **Approval Rating** (specifically, the `review_ratio` representing positive versus total reviews). By coupling the existing Ownership Tier classifier with this continuous regressor, a novel inference dashboard can be developed. This system would project new game prototypes onto a **2D Market Fit Matrix**:

- **X-Axis (Volume)**: Estimated player base and ownership tier (generated by the current XGBoost classifier).
- **Y-Axis (Sentiment)**: Estimated community approval rating (generated by the proposed auxiliary regressor).

This quadrant-based visualization would allow publishers and developers to immediately identify "High-Risk / Low-Reward" prototypes or predict "Niche Critical Darlings" (Low

Volume, High Sentiment), offering a holistic view of a game’s potential market fit.

5.2.1 Multi-Modal Expansion

Future iterations of the pipeline could also expand beyond textual and tabular metadata. Incorporating deep multi-modal architectures—such as Convolutional Neural Networks (CNNs) for analyzing promotional screenshots, or sequence models for processing gameplay trailer aesthetics—could capture the visual and atmospheric signals that heavily influence modern consumer purchasing behavior.

Bibliography

- [1] N. Davis, “Steam store games (clean dataset & raw data),” 2019, accessed for HitPredictor-Steam project. [Online]. Available: <https://www.kaggle.com/datasets/nikdavis/steam-store-raw>
- [2] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019. [Online]. Available: <https://aclanthology.org/D19-1410/>
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “Smote: synthetic minority over-sampling technique,” *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002. [Online]. Available: <https://www.jair.org/index.php/jair/article/view/10302>
- [4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631. [Online]. Available: <https://arxiv.org/abs/1907.10902>
- [5] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794. [Online]. Available: <https://arxiv.org/abs/1603.02754>
- [6] R. Storn and K. Price, “Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces,” *Journal of global optimization*, vol. 11, no. 4, pp. 341–359, 1997. [Online]. Available: <https://doi.org/10.1023/A:1008202821328>
- [7] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., 2017, pp. 4765–4774. [Online]. Available: <https://arxiv.org/abs/1705.07874>
- [8] M. T. Ribeiro, S. Singh, and C. Guestrin, “"Why should I trust you?": Explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144. [Online]. Available: <https://arxiv.org/abs/1602.04938>